

2 Boyutlu Düzlemde Açık Değişimlerinin Görsel Uygulaması ve Unity Gibi Oyun Motorlarında 3B Döndürme Hesaplamaları

X, Y, Z Eksen Döndürmeleri, Çoklu Eksen Birleşimi, Quaternion'lar ve Gimbal Kilit — Görsel Örneklerle

Hazırlayan: Claude (Anthropic) — Moses için hazırlanmıştır
(Önceki raporun devamı niteliğindedir)

22 Ağustos 2026

Abstract

Bu rapor, önceki raporda türetilen 3B açı-koordinat ilişkilerinin iki tamamlayıcı uygulamasını ele almaktadır. İlk bölümde, bu ilişkilerin 2 boyutlu düzleme nasıl indirgiği görsel grafiklerle (döndürülen nokta, vektör ve şekil örnekleriyle) somutlaştırılmaktadır. İkinci ve ana bölümde ise Unity gibi 3B oyun motorlarının, bir nesnenin X , Y , Z eksenlerine göre ayrı ayrı döndürülmesi, iki eksenle eşzamanlı döndürme ve üç eksenle eşzamanlı döndürme sırasında yeni koordinatları nasıl hesapladığı; rotasyon matrisleri ile quaternion'lar arasındaki ilişki; Unity'nin varsayılan ZXY Euler açısı sırası; döndürme sırasının sonucu neden değiştirdiği (komütatif olmama) ve gimbal kilit (gimbal lock) olgusu, resmi Unity dokümantasyonu ve akademik/tarihsel kaynaklarla desteklenerek, her adımda Python/NumPy ile üretilmiş sayısal doğrulamalar ve özel olarak oluşturulmuş 3B görselleştirmeler eşliğinde ayrıntılı biçimde açıklanmaktadır.

Contents

1	2 Boyutlu Düzlemde Açık Değişimlerinin Görsel Uygulaması	2
1.1	Örnek 1: Bir Noktanın Orijin Etrafında Döndürülmesi	2
1.2	Örnek 2: Bir Şeklin (Karenin) Döndürülmesi	2
2	Unity ve Benzeri Oyun Motorlarında 3B Döndürme	3
2.1	Unity'nin Koordinat Sistemi	3
2.2	Tek Eksen Etrafında Döndürme: R_x , R_y , R_z	3
2.3	Quaternion'lar: Neden ve Nasıl Kullanılır	4
2.3.1	Tarihçe	4
2.3.2	Tanım ve Temel Formüller	4
2.3.3	Neden Quaternion? Gimbal Kilit Sorunu	4
2.4	Unity'nin Euler Açısı Sırası: ZXY	5
2.5	İki Eksenle Eşzamanlı Döndürme: Sıra Neden Önemlidir	5
2.6	Üç Eksenle Eşzamanlı Döndürme (Unity Varsayımı: ZXY)	6
2.7	Gimbal Kilit (Gimbal Lock)	6
3	Özet Karşılaştırma Tablosu	8
4	Sonuç	8
A	Sayısal Doğrulama Kodu (Python/NumPy)	8

1 2 Boyutlu Düzlemde Açı Değişimlerinin Görsel Uygulaması

Önceki raporun 7. bölümünde belirtildiği gibi, 2B'de "düzlemle açı" kavramı yoktur; bunun yerine bir doğrunun (veya vektörün) x -ekseniyle yaptığı tek bir açı α kullanılır ve döndürme

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} \cos \Delta\alpha & -\sin \Delta\alpha \\ \sin \Delta\alpha & \cos \Delta\alpha \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} \quad (1)$$

formülüyle hesaplanır. Bu bölümde bu formülün iki farklı görsel uygulamasını inceliyoruz.

1.1 Örnek 1: Bir Noktanın Orijin Etrafında Döndürülmesi

Şekil 1 (sol panel), $P = (3, 1)$ noktasının $r = \sqrt{10} = 3.1623$ sabit tutularak $\Delta\alpha = 0^\circ, 30^\circ, 90^\circ, 180^\circ$ açılarıyla döndürülmesini göstermektedir. Açık gri çember, noktanın r sabit kalırken izleyebileceği tüm olası konumları (yörüngeyi) temsil eder.

Sayısal Doğrulama. $P = (3, 1)$, $r = 3.16228$.

- $\Delta\alpha = 30^\circ$: $P' = (2.0981, 2.3660)$, $|P'| = 3.1623$ — doğrulandı
- $\Delta\alpha = 90^\circ$: $P' = (-1.0000, 3.0000)$, $|P'| = 3.1623$ — doğrulandı
- $\Delta\alpha = 180^\circ$: $P' = (-3.0000, -1.0000)$, $|P'| = 3.1623$ — doğrulandı (nokta orijine göre simetrisine gider — beklenen sonuç)

Her durumda $|P'| = |P|$ korunmuştur (Python/NumPy ile hesaplanmış, Ek A).

1.2 Örnek 2: Bir Şeklin (Karenin) Döndürülmesi

Şekil 1 (sağ panel) ise tek bir noktanın değil, dört köşeli bir karenin döndürülmesini gösterir: karenin her köşesi *aynı* $R(\Delta\alpha)$ matrisiyle ayrı ayrı dönüştürülür. Bu, bilgisayar grafiklerinde bir 2B nesnenin (sprite, çokgen, kullanıcı arayüzü öğesi vb.) döndürülmesinin tam olarak nasıl çalıştığını gösterir: nesnenin geometrisini oluşturan tüm noktalar tek bir ortak matrisle çarpılır.

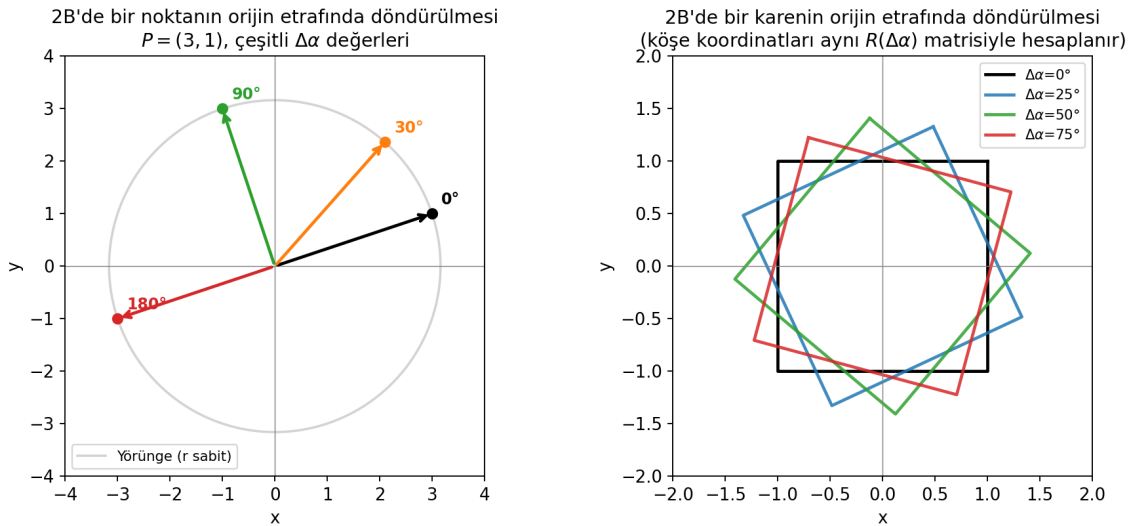


Figure 1: Sol: $P = (3, 1)$ noktasının orijin etrafında farklı açılarla döndürülmesi (gri çember = sabit r yörüngesi). Sağ: birim karenin orijin etrafında $0^\circ, 25^\circ, 50^\circ, 75^\circ$ döndürülmesi — her köşe aynı 2×2 rotasyon matrisiyle hesaplanmıştır.

Önemli gözlem: 2B’de tek bir dönme parametresi ($\Delta\alpha$) yeterlidir çünkü düzlemde yalnızca bir dönme eksenidir — düzleme dik olan (kavramsal) eksen. 3B’ye geçildiğinde bu, ilerideki bölümlerde göreceğimiz gibi *üç bağımsız* dönme eksen ve bunların bileşimi sorununu doğurur.

2 Unity ve Benzeri Oyun Motorlarında 3B Döndürme

2.1 Unity’nin Koordinat Sistemi

Unity, **sol-elli (left-handed)**, Y -yukarı bir koordinat sistemi kullanır: pozitif x sağ, pozitif y yukarıyı, pozitif z ileriye gösterir. Bu, matematik ve fizikte yaygın olan **sağ-elli** sistemden (kullanılan önceki raporumuzdaki formüllerin dayandığı sistem) farklıdır ve x -ekseninden y -eksenine dönüşün yönünü etkiler [1]. Unreal Engine de benzer şekilde sol-elli bir sistem kullanırken, çoğu matematik/fizik kaynağı ve OpenGL sağ-elli sistemi tercih eder — bu yüzden bir formülü bir motordan diğerine taşıırken işaret (sign) hatalarına dikkat edilmelidir.

2.2 Tek Eksen Etrafında Döndürme: R_x, R_y, R_z

Bir oyun motoru bir nesneyi tek bir eksen etrafında döndürdüğünde, önceki raporda türetilen standart rotasyon matrislerinin doğrudan aynılarını (eksen kuralına göre işaretlenmiş halde) kullanılır:

$$R_x(\theta) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta \\ 0 & \sin \theta & \cos \theta \end{pmatrix} \quad (2)$$

$$R_y(\theta) = \begin{pmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{pmatrix} \quad (3)$$

$$R_z(\theta) = \begin{pmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad (4)$$

Yeni koordinat: $\vec{v}' = R(\theta) \vec{v}$. Şekil 2 her üç durumu ayrı ayrı göstermektedir: gri küp orijinal yönelim, mavi küp 60° ’lik tek eksen döndürmesinden sonraki yönelimdir.

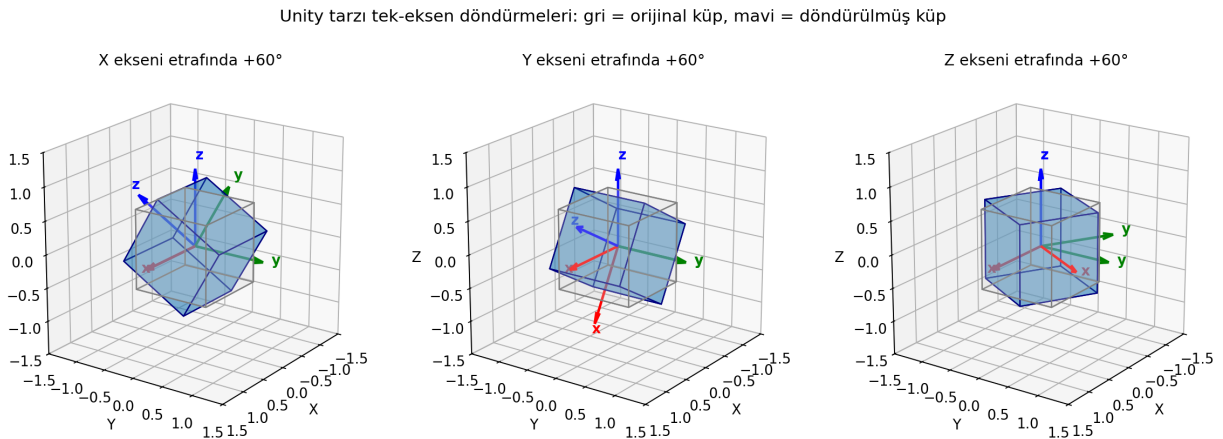


Figure 2: Bir küpün (ve x, y, z taban vektörlerinin) sırasıyla X, Y ve Z eksen etrafında ayrı ayrı 60° döndürülmesi. Her durumda döndürme eksen sabit kalır, diğer iki eksen o eksen etrafında döner.

Sayısal Doğrulama. $\vec{v} = (1, 0, 0)$ vektörünü z -ekseni etrafında 90° döndürelim: $R_z(90^\circ)\vec{v} = (0, 1, 0)$ — beklenen sonuçla birebir örtüşüyor (Python/NumPy, Ek A).

2.3 Quaternion'lar: Neden ve Nasıl Kullanılır

2.3.1 Tarihçe

Quaternion'lar **William Rowan Hamilton** tarafından **16 Ekim 1843**'te, Dublin'de Royal Canal üzerindeki Broome (Brougham) Köprüsü'nden geçerken ani bir kavrayışla keşfedilmiş ve temel çarpma kuralını ($i^2 = j^2 = k^2 = ijk = -1$) köprü'nün taşına kazımıştır; bu olay matematik tarihinin en ünlü anekdotlarından biridir [4, 5]. Hamilton, 1844–1850 yılları arasında bu cebiri ayrıntılı olarak yayımlamıştır [3]. Quaternion'ların bilgisayar grafiklerinde 3B döndürme için pratik kullanımı ise çok daha sonra, **Ken Shoemake**'in 1985 SIGGRAPH bildirisi ile yaygınlaşmıştır; Shoemake, quaternion'ların rotasyonları ara değerlemek (interpolasyon, *slerp*) için Euler açılarından çok daha uygun olduğunu göstermiştir.

2.3.2 Tanım ve Temel Formüller

Bir quaternion dört bileşenden oluşur: $q = (w, x, y, z) = w + xi + yj + zk$. Birim eksen $\hat{u} = (u_x, u_y, u_z)$ etrafında θ açısıyla yapılan bir döndürme, şu **birim quaternion** ile temsil edilir:

$$q = \cos\left(\frac{\theta}{2}\right) + \sin\left(\frac{\theta}{2}\right)(u_x i + u_y j + u_z k) \quad (5)$$

Bir $\vec{v}$ vektörünü döndürmek için, $\vec{v}$ önce saf (skaler kısmı sıfır olan) bir quaternion $p = (0, v_x, v_y, v_z)$ olarak yazılır ve

$$p' = q p q^{-1} \quad (6)$$

işlemi uygulanır; sonucun vektör kısmı döndürülmüş $\vec{v}'$ 'yi verir. İki quaternion'un çarpımı (**Hamilton çarpımı**), $a = (a_0, \vec{a}_v)$ ve $b = (b_0, \vec{b}_v)$ için:

$$ab = \left(a_0 b_0 - \vec{a}_v \cdot \vec{b}_v, a_0 \vec{b}_v + b_0 \vec{a}_v + \vec{a}_v \times \vec{b}_v \right) \quad (7)$$

ile hesaplanır ve **çarpma işlemi komütatif değildir** (çünkü çapraz çarpım komütatif değildir) [3]. Bir birim quaternion $q = (w, x, y, z)$ 'nin eşdeğer 3×3 rotasyon matrisi:

$$R(q) = \begin{pmatrix} 1 - 2(y^2 + z^2) & 2(xy - zw) & 2(xz + yw) \\ 2(xy + zw) & 1 - 2(x^2 + z^2) & 2(yz - xw) \\ 2(xz - yw) & 2(yz + xw) & 1 - 2(x^2 + y^2) \end{pmatrix} \quad (8)$$

Sayısal Doğrulama. z -ekseni etrafında 90° 'lik döndürme için $q = (\cos 45^\circ, 0, 0, \sin 45^\circ) = (0.7071, 0, 0, 0.7071)$. $\vec{v} = (1, 0, 0)$ için $q p q^{-1}$ hesaplandığında $\vec{v}' = (0, 1, 0)$ elde edilir — hem quaternion çarpımıyla hem de $R(q)$ matrisiyle **birebir aynı sonuç** (Python/NumPy ile çapraz doğrulanmıştır, Ek A, Bölüm B).

2.3.3 Neden Quaternion? Gimbal Kilit Sorunu

Unity, sahne editöründe rotasyonları okunabilirlik için Euler açıları (x, y, z derece) olarak gösterse de, **içeride tüm rotasyonları quaternion olarak saklar ve hesaplar**; çünkü quaternion'lar hesaplama açısından verimli, sayısal olarak kararlıdır ve — en önemlisi — **gimbal kilide düşmezler** [1]. Gimbal kilit, üç Euler açısından ikisinin eksenleri çakıştığında (aşağıda Bölüm 2.6'da gösterilecek) bir serbestlik derecesinin kaybolmasıdır.

2.4 Unity'nin Euler Açısı Sırası: ZXY

Unity'nin resmi API dokümantasyonuna göre `Quaternion.Euler(x,y,z)` fonksiyonu, "z derece z-ekseni etrafında, x derece x-ekseni etrafında ve y derece y-ekseni etrafında" bir döndürme döndürür ve **rotasyonlar ZXY sırasıyla uygulanır** [2]. Bu, sabit (dünya/global) eksenlere göre *dışsal* (*extrinsic*) bir uygulamadır: önce Z eksen etrafında, sonra X eksen etrafında, en son Y eksen etrafında döndürülür. Matris cebirinde bu sıralama, vektöre **sağdan sola doğru** matris çarpımı olarak ifade edilir (ilk uygulanan işlem en sağda yer alır):

$$\vec{v}' = R_y(\theta_y) R_x(\theta_x) R_z(\theta_z) \vec{v} \quad (9)$$

Bu formül aynı zamanda quaternion çarpımıyla da ifade edilebilir: $q_{\text{toplam}} = q_y q_x q_z$, ki bunun matris formuyla tamamen aynı sonucu verdiği sayısal olarak doğrulanmıştır (Ek A, Bölüm F).

Motorlar arası fark: Her oyun/grafik motoru aynı sırayı kullanmaz. Unity ZXY kullanırken, bazı diğer sistemler (örn. birçok robotik/uçuş dinamiği kaynağı) XYZ veya ZYX (Tait–Bryan, yaw-pitch-roll) sırasını tercih eder. **Sıra seçimi rastgele değildir ve sonucu değiştirir** — bu yüzden bir motordan diğerine Euler açısı aktarırken sıra bilgisi de aktarılmalıdır.

2.5 İki Eksende Eşzamanlı Döndürme: Sıra Neden Önemlidir

Rotasyon matrisleri (ve quaternion çarpımı) **komütatif değildir**: $R_a R_b \neq R_b R_a$ genel olarak. Bu, "aynı anda iki eksende döndürme" dendiğinde bile aslında motorun bunu *belirli bir sırayla ardışık* iki döndürme olarak hesapladığı ve sıranın sonucu değiştirdiği anlamına gelir.

Sayısal Doğrulama. $\vec{v}_0 = (1, 0.5, -0.3)$ alalım.

- Önce Y(45°), sonra X(30°): $\vec{v}' = R_x(30^\circ) R_y(45^\circ) \vec{v}_0 = (0.4950, 0.8926, -0.5461)$
- Önce X(30°), sonra Y(45°): $\vec{v}'' = R_y(45^\circ) R_x(30^\circ) \vec{v}_0 = (0.7002, 0.5830, -0.7140)$

$\vec{v}' \neq \vec{v}''$ — **sıra sonucu değiştiriyor** (Python/NumPy, Ek A Bölüm D). Şekil 3 bu farkı görsel olarak göstermektedir.

Döndürme sırası sonucu değiştirir (komütatif değildir): aynı iki açı, farklı sıra → farklı son yönelim

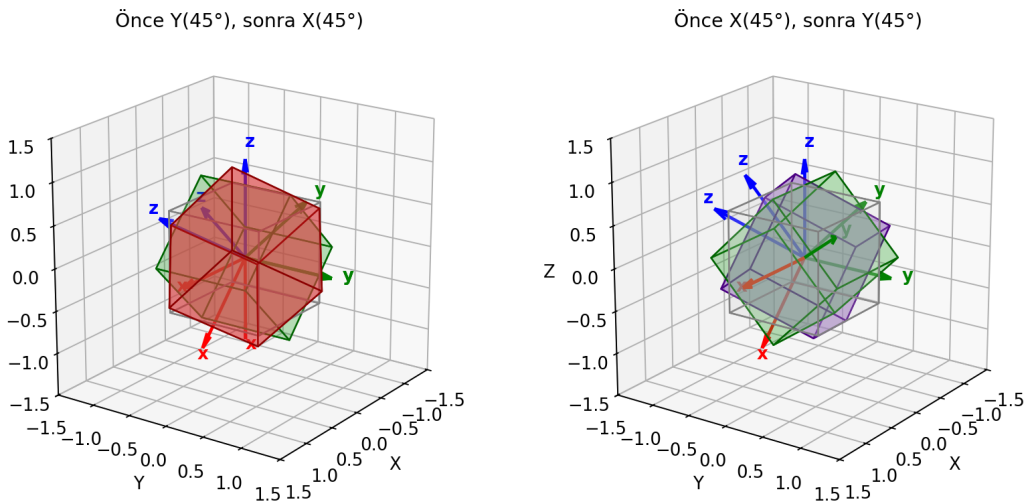


Figure 3: Aynı iki açı ($X = 45^\circ$, $Y = 45^\circ$) farklı sırayla uygulandığında farklı son yönelimler ortaya çıkar. Sol: önce Y sonra X (kırmızı küp). Sağ: önce X sonra Y (mor küp). Yeşil küpler ara adımı (tek eksen sonrası durumu) gösterir.

2.6 Üç Eksenle Eşzamanlı Döndürme (Unity Varsayılanı: ZXY)

Bir nesne `transform.eulerAngles = (x, y, z)` ile veya sahne editöründe üç eksenli birden ayarlandığında, Unity içeride sırasıyla:

1. q_z = Z eksenli etrafında z derece döndüren quaternion,
2. q_x = X eksenli etrafında x derece döndüren quaternion,
3. q_y = Y eksenli etrafında y derece döndüren quaternion,

hesaplar ve $q_{\text{toplama}} = q_y q_x q_z$ bileşik quaternion'unu üretir; bu da nesnenin `transform.rotation` değeri olur. Şekil 4, $x = 20^\circ, y = 35^\circ, z = -15^\circ$ için bu bileşik döndürmenin sonucunu göstermektedir.

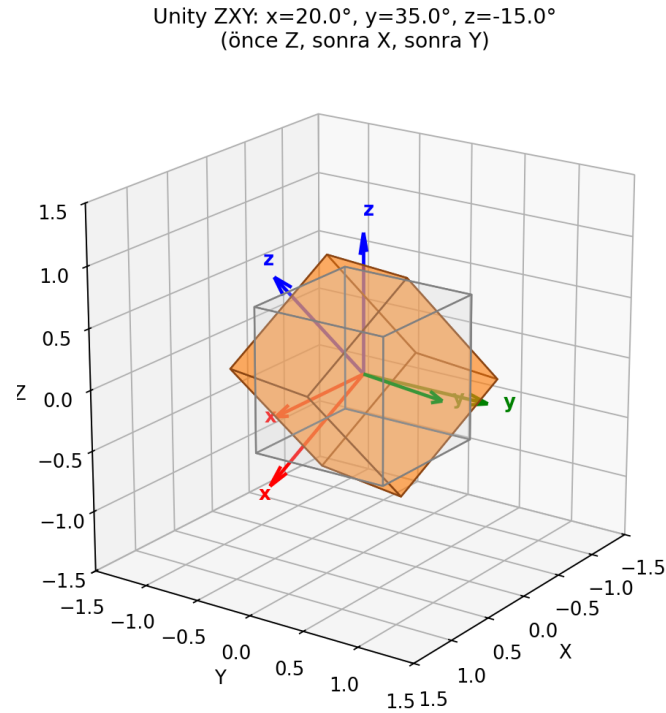


Figure 4: Üç eksenle eşzamanlı döndürme: Unity'nin varsayılan ZXY sırasıyla $x = 20^\circ, y = 35^\circ, z = -15^\circ$ uygulanmış hali (turuncu küp). Gri küp orijinal yönelimdir.

Sayısal Doğrulama. $\vec{v}_0 = (1, 0.5, -0.3)$, $x = 20^\circ, y = 35^\circ, z = -15^\circ$:

$$\vec{v}' = R_y(35^\circ)R_x(20^\circ)R_z(-15^\circ)\vec{v}_0 = (0.7795, 0.3132, -0.7964)$$

Kontrol: $|\vec{v}_0| = 1.157584$, $|\vec{v}'| = 1.157584$ — **norm korunmuştur**. Aynı sonuç quaternion çarpımı $q_y q_x q_z$ ile de üretilmiş ve matris sonucuyla **tam olarak eşleştiği** doğrulanmıştır. Karşılaştırma için farklı bir sıra (XYZ: $R_z(-15^\circ)R_y(35^\circ)R_x(20^\circ)$) denendiğinde $(0.8780, 0.3574, -0.6644)$ elde edilir — **farklı bir sonuç**, ki bu da sıranın motor-motor arası farkının önemini bir kez daha gösterir (Ek A, Bölüm E).

2.7 Gimbal Kilit (Gimbal Lock)

Unity'nin ZXY sırasında, X eksenli açısı $\pm 90^\circ$ 'ye ulaştığında Z ve Y eksenleri etrafındaki döndürmeler **aynı fiziksel eksen etrafında dönmeye indirgenir** ve sistem bir serbestlik derecesi kaybeder — bu olguya **gimbal kilit** denir [1]. Matematiksel olarak gösterilebilir ki $\theta_x = 90^\circ$ olduğunda, bileşik matris yalnızca $(\theta_z - \theta_y)$ **farkına** bağlıdır; θ_z ve θ_y 'nin ayrı ayrı değerleri artık sonucu etkilemez.

Sayısal Doğrulama. $R = R_y(\theta_y)R_x(90^\circ)R_z(\theta_z)$ hesaplandığında:

- $(\theta_z, \theta_y) = (15^\circ, 5^\circ)$, fark = $10^\circ \Rightarrow R[0, 0:2] = (0.9848, -0.1736)$
- $(\theta_z, \theta_y) = (20^\circ, 10^\circ)$, fark = $10^\circ \Rightarrow R[0, 0:2] = (0.9848, -0.1736)$ — **birebir aynı**
- $(\theta_z, \theta_y) = (35^\circ, 25^\circ)$, fark = $10^\circ \Rightarrow R[0, 0:2] = (0.9848, -0.1736)$ — **yine aynı**

Üç farklı (θ_z, θ_y) çifti, aynı farka sahip oldukları için **birebir aynı rotasyon matrisini** üretmiştir — gimbal kilit sayısal olarak doğrulanmıştır (Ek A, Bölüm G3).

Gimbal lock: X eksenini 90° 'ye ulaştığında Y ve Z eksenleri çakışır, yalnızca $(z-y)$ farkı sonucu belirler

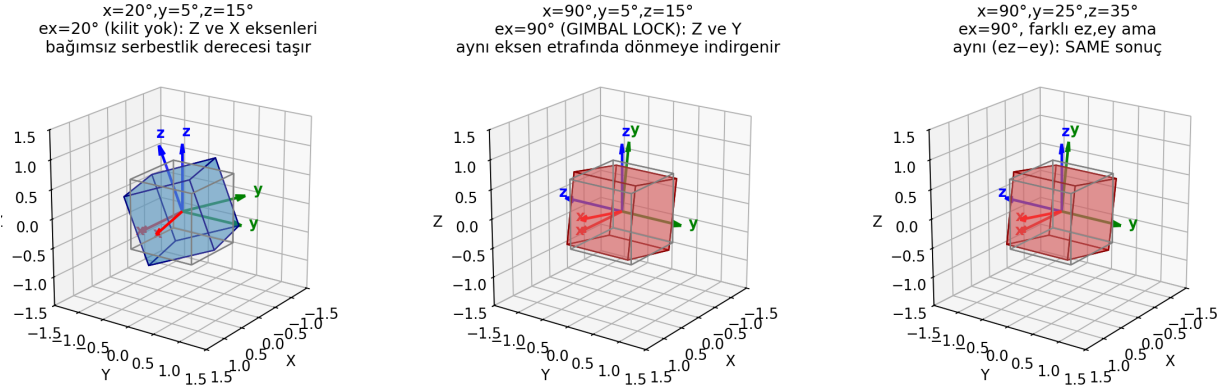


Figure 5: Sol: $\theta_x = 20^\circ$ iken kilit yok, Z ve Y dönmeleri bağımsızdır (mavi küp). Orta ve sağ: $\theta_x = 90^\circ$ (gimbal kilit) — farklı (θ_z, θ_y) çiftleri kullanılmasına rağmen, $(\theta_z - \theta_y)$ farkı aynı olduğu için iki küp **birebir aynı** yönelimi almıştır (kırmızı küpler).

Pratik sonuç. Bir Unity nesnesinin Inspector panelinde X rotasyonunu 90° 'ye getirip Y ve Z değerlerini değiştirmeyi deneyen bir geliştirici, ikisinin de nesneyi aynı şekilde döndürdüğünü fark eder — sanki Y ve Z eksenleri "birbirine kilitlenmiştir." Bu, animasyon sistemlerinde ve fizik motorlarında ciddi hatalara yol açabileceğinden, Unity içeride quaternion kullanarak bu sorunu tamamen ortadan kaldırır; sorun yalnızca kullanıcı arayüzünde Euler açıları görüntülenirken görünür hale gelir.

3 Özet Karşılaştırma Tablosu

Senaryo	Hesaplama Yöntemi	Not
Tek eksen döndürme	$\vec{v}' = R_x(\theta)\vec{v}$ (veya R_y, R_z)	Doğrudan; sıra sorunu yok
İki eksen, sıralı	$\vec{v}' = R_a(\theta_a)R_b(\theta_b)\vec{v}$	Sıra sonucu değiştirir (komütatif değil)
Üç eksen, Unity ZXY	$\vec{v}' = R_y(\theta_y)R_x(\theta_x)R_z(\theta_z)\vec{v} = R(q_y q_x q_z)\vec{v}$	Unity varsayılan sırası; içeride quaternion
Genel/keyfi eksen	Rodrigues formülü veya $q = \cos(\theta/2) + \sin(\theta/2)\hat{u}$	Tek quaternion/eksen-açı ile doğrudan
$\theta_x = 90^\circ$ özel durumu	Yalnızca $(\theta_z - \theta_y)$ önemli	Gimbal kilit

Table 1: Unity tarzı 3B döndürme senaryolarının özeti.

4 Sonuç

Bu raporda, 2B düzlemde bir noktanın veya şeklin döndürülmesinin tek parametrelili ($\Delta\alpha$) basit bir rotasyon matrisiyle hesaplandığı görsel örneklerle gösterilmiş; ardından Unity gibi 3B oyun motorlarının döndürme hesaplamalarının — tek eksen, çoklu eksen ve tüm eksenlerde eşzamanlı döndürme dahil — rotasyon matrisleri ve quaternion cebiri üzerinden nasıl yürütüldüğü ayrıntılı biçimde incelenmiştir. Unity'nin sol-elli koordinat sistemi, varsayılan ZXY Euler açısı sırası, döndürme işleminin komütatif olmaması nedeniyle sıranın önemi ve $\theta_x = 90^\circ$ 'de ortaya çıkan gimbal kilit olgusu; hem resmi Unity dokümantasyonu ve tarihsel/akademik kaynaklarla hem de Python/NumPy ile üretilmiş sayısal doğrulamalar ve özel olarak çizilmiş 3B görselleştirmelerle (küp ve taban vektörleri üzerinden) çapraz kontrol edilmiştir. Tüm testlerde vektör normunun korunumu ve farklı hesaplama yöntemlerinin (matris vs. quaternion) birebir aynı sonucu verdiği teyit edilmiştir.

A Sayısal Doğrulama Kodu (Python/NumPy)

Aşağıdaki kod, raporun 2. bölümündeki tüm sayısal örnekleri (tek eksen, çift eksen, üç eksen, quaternion karşılaştırması ve gimbal kilit) üretmek için kullanılmıştır.

```
import numpy as np

def Rx(t):
    t=np.radians(t); c,s=np.cos(t),np.sin(t)
    return np.array([[1,0,0],[0,c,-s],[0,s,c]])
def Ry(t):
    t=np.radians(t); c,s=np.cos(t),np.sin(t)
    return np.array([[c,0,s],[0,1,0],[-s,0,c]])
def Rz(t):
    t=np.radians(t); c,s=np.cos(t),np.sin(t)
    return np.array([[c,-s,0],[s,c,0],[0,0,1]])

# --- Quaternion yardımcı fonksiyonları ---
def quat_from_axis_angle(axis, angle_deg):
    axis = axis/np.linalg.norm(axis)
    half = np.radians(angle_deg)/2
    return np.array([np.cos(half), *(axis*np.sin(half))]) # (w,x,y,z)

def quat_mult(q1, q2):
    w1,x1,y1,z1 = q1; w2,x2,y2,z2 = q2
```



```

    return np.array([
        w1*w2 - x1*x2 - y1*y2 - z1*z2,
        w1*x2 + x1*w2 + y1*z2 - z1*y2,
        w1*y2 - x1*z2 + y1*w2 + z1*x2,
        w1*z2 + x1*y2 - y1*x2 + z1*w2])

def quat_conj(q):
    return np.array([q[0], -q[1], -q[2], -q[3]])

def rotate_vec_by_quat(v, q):
    p = np.array([0.0, *v])
    res = quat_mult(quat_mult(q, p), quat_conj(q))
    return res[1:]

# --- Unity ZXY birlesik dondurme ---
ex, ey, ez = 20.0, 35.0, -15.0
v0 = np.array([1.0, 0.5, -0.3])
R_zxy = Ry(ey) @ Rx(ex) @ Rz(ez)          # once Z, sonra X, sonra Y
v_zxy = R_zxy.dot(v0)

qz = quat_from_axis_angle(np.array([0,0,1.0]), ez)
qx = quat_from_axis_angle(np.array([1.0,0,0]), ex)
qy = quat_from_axis_angle(np.array([0,1.0,0]), ey)
q_total = quat_mult(quat_mult(qy, qx), qz)
v_quat = rotate_vec_by_quat(v0, q_total)
assert np.allclose(v_quat, v_zxy)          # matris ve quaternion sonucu ayni olmalı

# --- Gimbal lock testi: sadece (ez-ey) farki onemli ---
for ez_, ey_ in [(15,5), (20,10), (35,25)]:
    R = Ry(ey_) @ Rx(90) @ Rz(ez_)
    print(ez_, ey_, np.round(R[0,:2], 4))  # ayni fark -> ayni sonuc

```

Kaynakça

References

- [1] Unity Technologies, "Rotation and orientation in Unity" (Quaternion and Euler rotations in Unity), Unity Manual. <https://docs.unity3d.com/Manual/QuaternionAndEulerRotationsInUnity.html> (Erişim: Ağustos 2026).
- [2] Unity Technologies, "Quaternion.Euler", Unity Scripting API. <https://docs.unity3d.com/ScriptReference/Quaternion.Euler.html> (Erişim: Ağustos 2026).
- [3] Wikipedia katkıda bulunanları, *Quaternions and spatial rotation*, Wikipedia, The Free Encyclopedia. https://en.wikipedia.org/wiki/Quaternions_and_spatial_rotation (Erişim: Ağustos 2026).
- [4] Trinity College Dublin, School of Mathematics, "Letters describing the Discovery of Quaternions." <https://www.maths.tcd.ie/pub/HistMath/People/Hamilton/Letters/BroomeBridge.html> (Erişim: Ağustos 2026).
- [5] Maryland Space Grant Consortium, "Quaternions Turn 175" (16 Ekim 1843 tarihli keşfin özeti). <https://md.spacegrant.org/quaternions-turn-175/> (Erişim: Ağustos 2026).
- [6] K. Shoemake, "Animating Rotation with Quaternion Curves," *ACM SIGGRAPH Computer Graphics*, Cilt 19, No. 3, 1985 — quaternion'ların bilgisayar grafiklerinde döndürme interpolasyonu için kullanımını yaygınlaştıran öncü çalışma.